

230449N

September 14, 2026

1 EN3160 - Image Processing and Machine Vision

1.1 Assignment 1: Intensity Transformations and Neighborhood Filtering

Student Name: Nuwanaka WAS

Index Number: 230449N

GitHub Profile: <https://github.com/samirunuwanaka>

Repo Link: <https://github.com/samirunuwanaka/ComputerVision/blob/main/Assignment/230449N.ipynb>

1.2 Setup and Dependencies

```
[27]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
import os
# Compact plots for PDF export
plt.rcParams.update({ 'figure.dpi': 72, 'savefig.dpi': 72, 'font.size': 7,
    ↪ 'axes.titlesize': 8, 'axes.labelsize': 7, 'xtick.labelsize': 6, 'ytick.
    ↪ labelsize': 6, 'figure.figsize': (5, 3), 'figure.autolayout': True, })
# Path configurations
IMAGE_DIR = 'images/images'
# Helper function for image display
def show_image(img, title='Image', cmap=None):
    plt.figure(figsize=(4.0, 2.5))
    if len(img.shape) == 3 and cmap is None:
        plt.imshow(cv.cvtColor(img, cv.COLOR_BGR2RGB))
    else:
        plt.imshow(img, cmap=cmap if cmap else 'gray')
    plt.title(title)
    plt.axis('off')
    plt.show()
print("OpenCV Version:", cv.__version__)
```

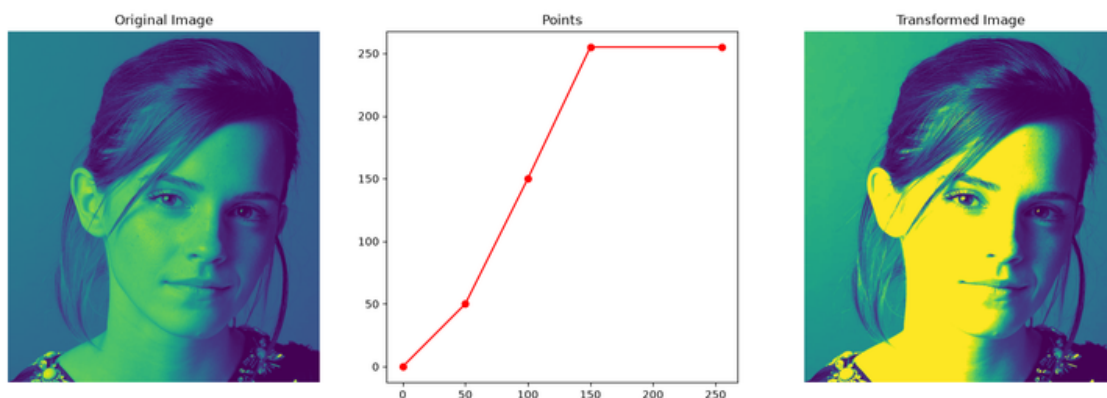
OpenCV Version: 5.0.0

1.3 Question 1: Piecewise Linear Intensity Transformation

Piecewise linear map `intensity_transform(im, breakpoints)` ($x \rightarrow y$), e.g.

$$\text{breakpoints} = \begin{bmatrix} 0 & 0 \\ 50 & 50 \\ 100 & 150 \\ 150 & 255 \\ 255 & 255 \end{bmatrix}$$

```
[28]: def intensity_transform(im: np.ndarray, breakpoints: np.ndarray) -> np.ndarray:
        """Applies piecewise linear intensity transformation to an input image.
        ↪Parameters: im (np.ndarray): Input image (grayscale...)"""
        return np.interp(im, breakpoints[:, 0], breakpoints[:, 1]).astype(np.uint8)
    im1 = cv.imread(os.path.join(IMAGE_DIR, 'q1.jpeg'), cv.IMREAD_GRAYSCALE)
    pts = np.array([[0, 0], [50, 50], [100, 150], [150, 255], [255, 255]])
    im1_trans = intensity_transform(im1, pts)
    fig, ax = plt.subplots(1, 3, figsize=(8.2, 2.8))
    # Original
    ax[0].imshow(im1); ax[0].set_title('Original Image'); ax[0].axis('off')
    # Points
    ax[1].plot(pts[:, 0], pts[:, 1], 'ro-')
    ax[1].set_title('Points')
    ax[1].set_aspect('equal')
    # Transformed
    ax[2].imshow(im1_trans); ax[2].set_title('Transformed Image'); ax[2].axis('off')
    plt.tight_layout()
    plt.show()
```



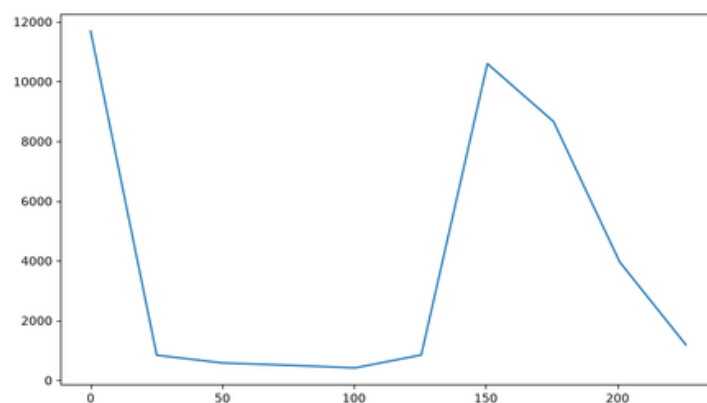
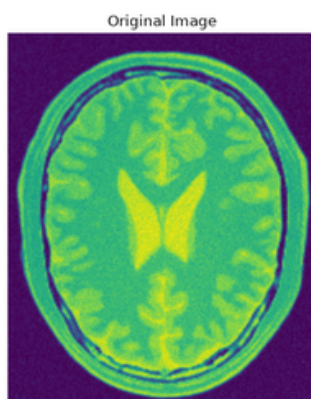
1.3.1 Discussion (Q1)

The transformed image has fewer shadows. High-contrast regions set to a constant appear bright; after intensity 50 the boost increases overall contrast.

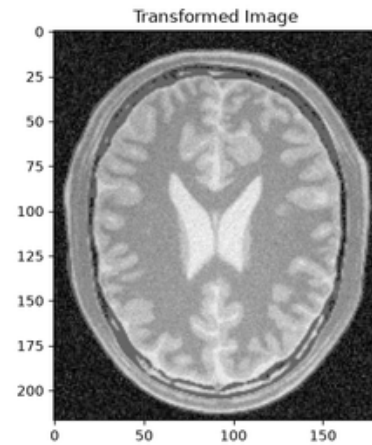
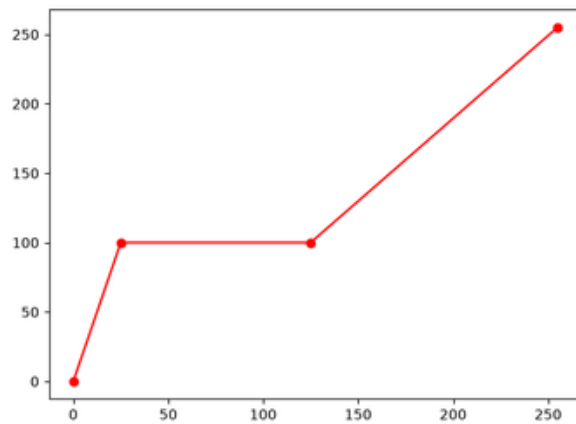
1.4 Question 2: Tissue Accentuation (Brain PD Image)

Accentuate (a) white matter and (b) gray matter on im02.png with custom breakpoint curves.

```
[29]: im2 = cv.imread(os.path.join(IMAGE_DIR, 'q2.jpeg'), cv.IMREAD_GRAYSCALE)
fig, ax = plt.subplots(1, 2, figsize=(8.2, 2.8))
# Original
ax[0].imshow(im2); ax[0].set_title('Original Image'); ax[0].axis('off')
# Points
hist = np.histogram(im2.flatten())
ax[1].plot(hist[1][:-1], hist[0])
plt.tight_layout()
plt.show()
pts = np.array([ [0, 0], [25, 100], [125, 100], [255, 255] ])
print("breakpoints of intensity transfer function are",*pts)
# Transformed
fig, ax = plt.subplots(1, 2, figsize=(8.2, 2.8))
ax[0].plot(pts[:,0],pts[:,1],'ro-')
ax[1].imshow(intensity_transform(im2, pts), cmap='gray')
ax[1].set_title('Transformed Image')
plt.show()
# cdf = hist.cumsum()
# # Ignore zero values in the CDF
# cdf_min = cdf[cdf > 0][0]
# # Histogram equalization mapping
# mapping = ((cdf - cdf_min) / (im2.size - cdf_min) * 255).clip(0, 255).
#         ↪ astype(np.uint8)
# plt.plot(range(256), mapping)
# plt.xlabel('Original intensity')
# plt.ylabel('Equalized intensity')
# plt.title('Histogram Equalization Transformation')
# plt.grid()
# plt.show()
```



breakpoints of intensity transfer function are [0 0] [25 100] [125 100] [255 255]



1.4.1 Discussion (Q2)

The histogram has an unused intensity range; stretching gray/white matter into that range increases intensity resolution and separates the tissues.

1.5 Question 3: Gamma Correction in $L^*a^*b^*$

Apply gamma to the L^* plane of im03.png:

$$L_{\text{out}} = 255 \left(\frac{L_{\text{in}}}{255} \right)^\gamma$$

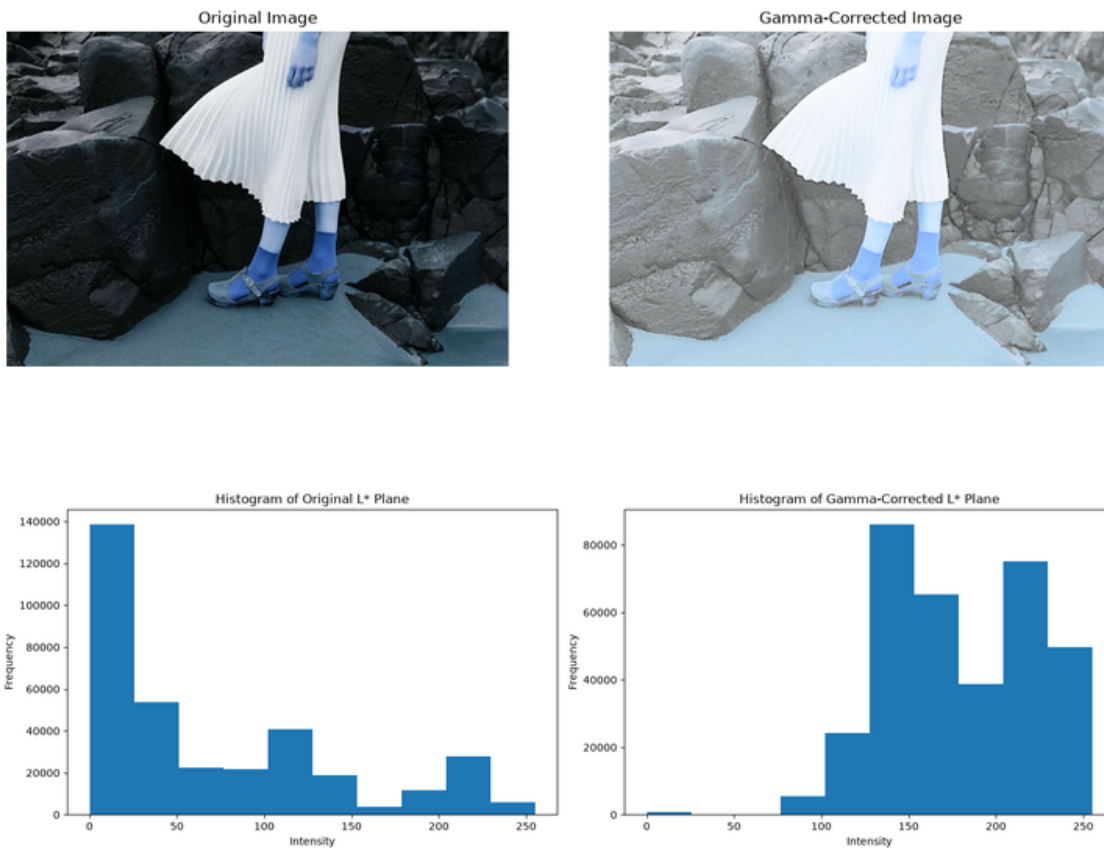
```
+b* Yellow
^
-a* Green <- * -> +a* Red
v
-b* Blue
```

```
[30]: def gamma_correct_L_plane(im_bgr: np.ndarray, gamma: float) -> tuple[np.
      ↪ ndarray, np.ndarray, np.ndarray]:
      """Applies gamma correction to the L* plane in L*a*b* color space.
      ↪ Parameters: im_bgr (np.ndarray): BGR image gamma (flo..."""
      imLAB = cv.cvtColor(im_bgr, cv.COLOR_BGR2LAB)
      L_in = np.array(imLAB[:, :, 0], dtype=np.uint8)
      L_out = 255 * ((L_in / 255) ** gamma)
      imLAB[:, :, 0] = L_out
      return cv.cvtColor(imLAB, cv.COLOR_LAB2BGR), L_in, L_out
      im3 = cv.imread(os.path.join(IMAGE_DIR, 'WhatsApp Image 2026-09-09 at 20.02.44_
      ↪ (1).jpeg'))
      gamma_val = 0.2 # Choose optimal gamma
      im3_corrected, L_orig, L_corr = gamma_correct_L_plane(im3, gamma_val)
```

```

fig, ax = plt.subplots(1, 2, figsize=(8.2, 2.8))
ax[0].imshow(im3); ax[0].set_title('Original Image'); ax[0].axis('off')
ax[1].imshow(im3_corrected); ax[1].set_title('Gamma-Corrected Image'); ax[1].
    ↪axis('off')
plt.show()
fig, ax = plt.subplots(1, 2, figsize=(8.2, 2.8))
ax[0].hist(L_orig.flatten()); ax[0].set_title('Histogram of Original L* Plane')
ax[0].set_xlabel('Intensity')
ax[0].set_ylabel('Frequency')
ax[1].hist(L_corr.flatten()); ax[1].set_title('Histogram of Gamma-Corrected L*
    ↪Plane')
ax[1].set_xlabel('Intensity')
ax[1].set_ylabel('Frequency')
plt.tight_layout()
plt.show()

```



1.5.1 Discussion (Q3)

In $L^*a^*b^*$, gamma is applied only to L^* , so luminance is corrected without separately gamma-correcting each RGB channel.

1.6 Question 4: Vibrance Enhancement (HSV)

On im04.jpg, enhance S with

$$f(x) = \min \left(x + a \cdot 128 e^{-\frac{(x-128)^2}{2\sigma^2}}, 255 \right), \quad \sigma = 70$$

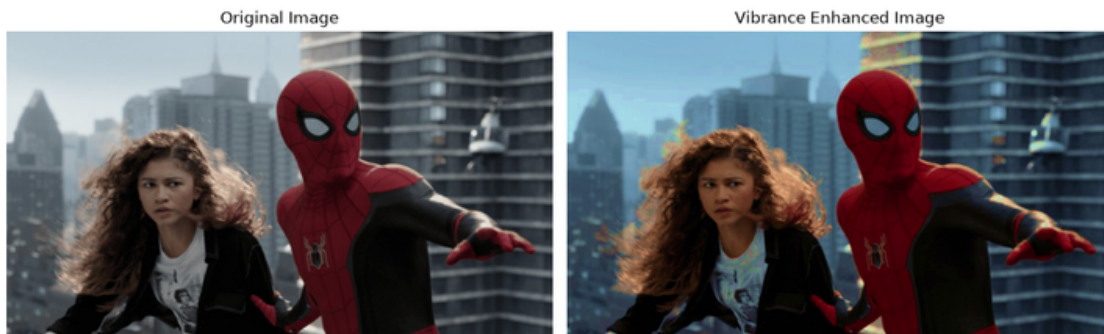
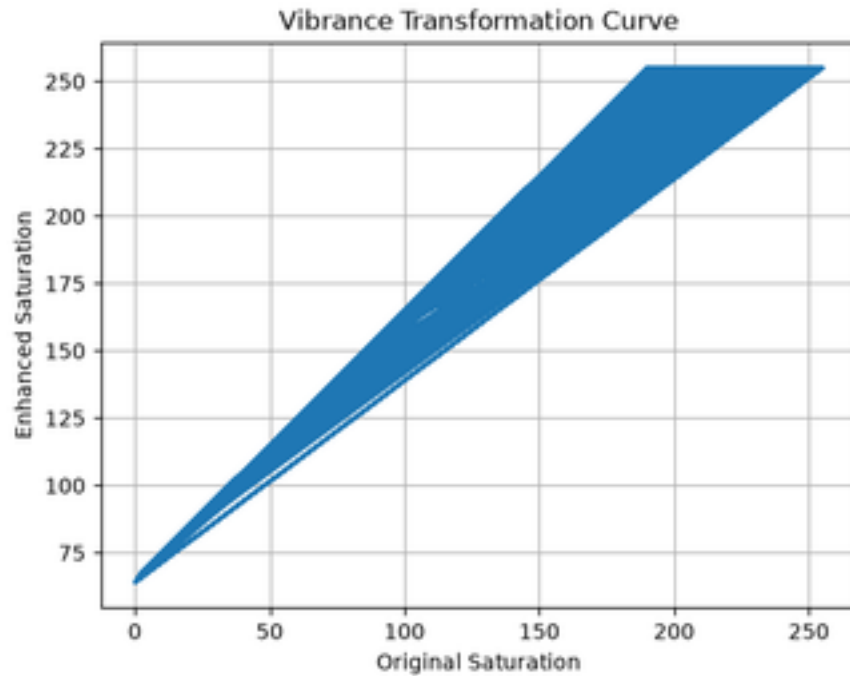
```
[31]: def vibrance_transform(S_plane: np.ndarray, a: float, sigma: float = 70.0) -> np.ndarray:
        """Applies vibrance enhancement function to Saturation channel."""
        return np.array([ np.uint8(min( x + a * 128 * np.exp(-((x - 128)**2) / (2 * sigma**2)), 255 )) for x in S_plane.flatten() ]).reshape(S_plane.shape)
im4 = cv.imread(os.path.join(IMAGE_DIR, 'WhatsApp Image 2026-09-09 at 20.02.441.jpeg'))
hsv = cv.cvtColor(im4, cv.COLOR_BGR2HSV)
H, S, V = cv.split(hsv)
a_val = 0.5 # Tune value
S_enh = vibrance_transform(S, a=a_val)
hsv_enh = cv.merge([H, S_enh, V])
im4_enh = cv.cvtColor(hsv_enh, cv.COLOR_HSV2BGR)
plt.plot(S.flatten(), S_enh.flatten())
plt.title('Vibrance Transformation Curve')
plt.xlabel('Original Saturation')
plt.ylabel('Enhanced Saturation')
plt.grid()
plt.show()
plt.figure(figsize=(6.6, 3.3))
plt.subplot(1, 2, 1)
plt.imshow(cv.cvtColor(im4, cv.COLOR_BGR2RGB))
plt.title('Original Image')
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(cv.cvtColor(im4_enh, cv.COLOR_BGR2RGB))
plt.title('Vibrance Enhanced Image')
plt.axis('off')
plt.tight_layout()
plt.show()
```

D:\Temp\ipykernel_20228\1495923243.py:7: RuntimeWarning: overflow encountered in scalar subtract

```
    x + a * 128 * np.exp(-((x - 128)**2) / (2 * sigma**2)),
```

D:\Temp\ipykernel_20228\1495923243.py:7: RuntimeWarning: overflow encountered in scalar negative

```
    x + a * 128 * np.exp(-((x - 128)**2) / (2 * sigma**2)),
```



1.6.1 Discussion (Q4)

Gaussian weighting boosts mid-range saturation most and less at extremes, reducing clipping in already-vibrant colours versus a uniform saturation boost.

1.7 Question 5: Custom Histogram Equalization

Manual HE on im05.jpg: histogram $\rightarrow$ PDF $\rightarrow$ CDF $\rightarrow s_k = \text{round}(255 \cdot T(r_k))$ (no `cv.equalizeHist`).

[32] :

```

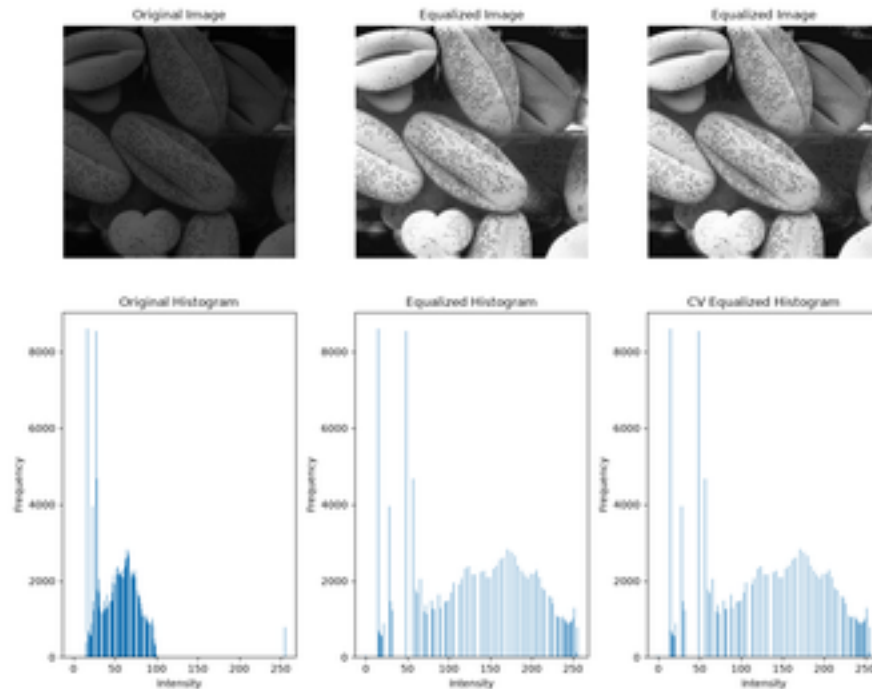
def custom_histogram_equalization(img_gray: np.ndarray) -> tuple[np.ndarray, np.
    ndarray, np.ndarray]:
    """Computes histogram equalization manually. Returns: tuple:
    (equalized_image, orig_hist, eq_hist)"""
    # 1. Calculate histogram
    intensity_frequency = np.array( [[i, np.count_nonzero(img_gray == i)] for i
    in range(256)] )
    # Original histogram
    orig_hist = intensity_frequency[:, 1].copy()
    # 2. Convert frequency to probability
    probability = intensity_frequency[:, 1] / np.sum(intensity_frequency[:, 1])
    # 3. Calculate CDF
    cdf = np.cumsum(probability)
    # 4. Scale CDF to [0, 255]
    mapping = np.round(255 * cdf).astype(np.uint8)
    # 5. Map original image intensities to new intensities
    equalized_image = mapping[img_gray]
    # 6. Calculate equalized histogram
    eq_hist = np.bincount( equalized_image.ravel(), minlength=256 )
    return equalized_image, orig_hist, eq_hist
im5 = cv.imread(os.path.join(IMAGE_DIR, 'image.png'), cv.IMREAD_GRAYSCALE)
im5_eq, hist_orig, hist_eq = custom_histogram_equalization(im5)
im5_eq_cv = cv.equalizeHist(im5)
im5_eq_cv_hist = np.array( [[i, np.count_nonzero(im5_eq_cv == i)] for i in
    range(256)] )
fig, ax = plt.subplots(2, 3, figsize=(6.6, 5.5))
# Original Image
ax[0, 0].imshow(im5, cmap='gray'); ax[0, 0].set_title('Original Image'); ax[0,
    0].axis('off')
# Original Histogram
ax[1, 0].bar(range(256), hist_orig)
ax[1, 0].set_title('Original Histogram')
ax[1, 0].set_xlabel('Intensity')
ax[1, 0].set_ylabel('Frequency')
# Equalized Image
ax[0, 1].imshow(im5_eq, cmap='gray'); ax[0, 1].set_title('Equalized Image');
    ax[0, 1].axis('off')
# Equalized Histogram
ax[1, 1].bar(range(256), hist_eq)
ax[1, 1].set_title('Equalized Histogram')
ax[1, 1].set_xlabel('Intensity')
ax[1, 1].set_ylabel('Frequency')
# Equalized Image
ax[0, 2].imshow(im5_eq_cv, cmap='gray'); ax[0, 2].set_title('Equalized Image');
    ax[0, 2].axis('off')
# Equalized Histogram

```

```

ax[1, 2].bar(range(256), im5_eq_cv_hist[:, 1])
ax[1, 2].set_title('CV Equalized Histogram')
ax[1, 2].set_xlabel('Intensity')
ax[1, 2].set_ylabel('Frequency')
plt.tight_layout()
plt.show()

```



1.7.1 Discussion (Q5)

Manual and OpenCV equalized histograms match.

1.8 Question 6: Selective Foreground Histogram Equalization

Equalize only the foreground (HSV threshold mask) of the given image; recombine with background.

```

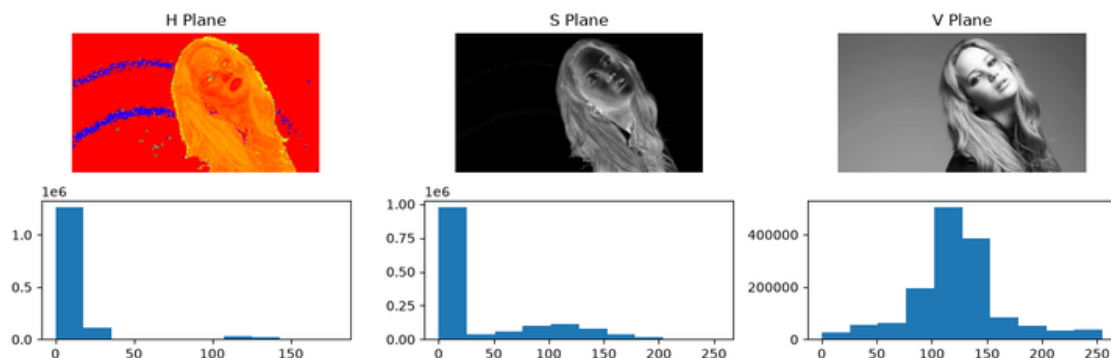
[ ]: im6 = cv.imread(os.path.join(IMAGE_DIR, 'WhatsApp Image 2026-09-09 at 20.02.44_
    ↪(2).jpeg'))
H, S, V = cv.split(cv.cvtColor(im6, cv.COLOR_BGR2HSV))
plt.figure(figsize=(6.6, 2.5))
plt.subplot(2, 3, 1)
plt.imshow(H, cmap='hsv')
plt.title('H Plane')
plt.axis('off')
plt.subplot(2, 3, 4)
plt.hist(H.flatten())

```

```

plt.subplot(2, 3, 2)
plt.imshow(S, cmap='gray')
plt.title('S Plane')
plt.axis('off')
plt.subplot(2, 3, 5)
plt.hist(S.flatten())
plt.subplot(2, 3, 3)
plt.imshow(V, cmap='gray')
plt.title('V Plane')
plt.axis('off')
plt.subplot(2, 3, 6)
plt.hist(V.flatten())
plt.tight_layout()
plt.show()
print("We can use S plane to mask out the background and then equalize the_
      foreground intensities")
# Use S channel to identify foreground
mask = cv.inRange(S, 15, 255)
# Extract foreground from V channel
foreground = cv.bitwise_and(V, V, mask=mask)
# Display intermediate results and final result
plt.figure(figsize=(8.2, 5.5))
plt.subplot(1, 3, 1)
plt.imshow(V, cmap='gray')
plt.title('V Plane')
plt.axis('off')
plt.subplot(1, 3, 2)
plt.imshow(foreground, cmap='gray')
plt.title('Masked Foreground')
plt.axis('off')
plt.subplot(1, 3, 3)
plt.imshow(foreground_eq, cmap='gray')
plt.title('Equalized Foreground')
plt.axis('off')
plt.tight_layout()
plt.show()

```



We can use S plane to mask out the background and then equalize the foreground intensities

We can use S plane to mask out the background and then equalize the foreground intensities



1.8.1 Discussion (Q6)

S plane used for foreground mask. Selective HE improves foreground contrast without globally stretching the background.

1.9 Question 7: Sobel Filtering & Separable Filtering

Sobel on q2.jpeg via (a) `cv.filter2D`, (b) manual 2D convolution, (c) separable 1D filters:

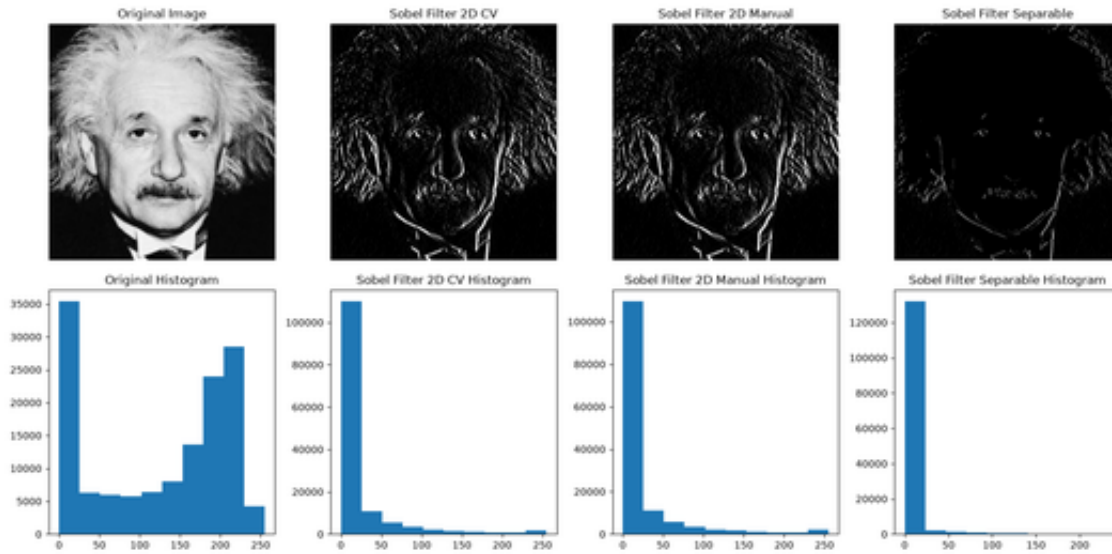
$$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}$$

```
[42]: sobel_filter_x = np.array([1, 0, -1])
sobel_filter_y = np.array([1, 2, 1])
def sobel_filter2d_cv(im: np.ndarray) -> np.ndarray:
    """(a) Sobel filtering using cv.filter2D"""
    sobel_filter = np.outer(sobel_filter_y, sobel_filter_x)
    return cv.filter2D( im, -1, sobel_filter )
def sobel_filter2d_manual(im: np.ndarray) -> np.ndarray:
    """(b) Sobel filtering using manual 2D convolution loop"""
    # Create 2D Sobel kernel
    kernel = np.outer(sobel_filter_y, sobel_filter_x)
    # Padding size for 3x3 kernel
    pad = kernel.shape[0] // 2
    # Pad image with zeros
    padded = np.pad( im, ((pad, pad), (pad, pad)), mode='constant',
    ↪constant_values=0 )
    # Output image
    output = np.zeros_like(im, dtype=np.float64)
    # Manual convolution
```

```

for i in range(im.shape[0]):
    for j in range(im.shape[1]):
        # Extract 3x3 neighbourhood
        region = padded[ i:i + kernel.shape[0], j:j + kernel.shape[1] ]
        # Convolution
        output[i, j] = np.sum(region * kernel)
    # Clip to valid image range
    output = np.clip(output, 0, 255)
return output.astype(np.uint8)
def sobel_filter_separable(im: np.ndarray) -> np.ndarray:
    """(c) Sobel filtering using 1D separable kernel decomposition"""
    # First apply vertical smoothing [1, 2, 1]^T
    temp = cv.filter2D( im, -1, sobel_filter_y.reshape(-1, 1) )
    # Then apply horizontal derivative [1, 0, -1]
    output = cv.filter2D( temp, -1, sobel_filter_x.reshape(1, -1) )
    return output
im7 = cv.imread(os.path.join(IMAGE_DIR, 'WhatsApp Image 2026-09-09 at 20.02.43_
↳(2).jpeg'), cv.IMREAD_GRAYSCALE)
fig, ax = plt.subplots(2, 4, figsize=(8.8, 4.4))
# Images
ax[0, 0].imshow(im7, cmap='gray'); ax[0, 0].set_title('Original Image'); ax[0, 0].axis('off')
ax[0, 1].imshow(sobel_filter2d_cv(im7), cmap='gray'); ax[0, 1].set_title('Sobel_
↳Filter 2D CV'); ax[0, 1].axis('off')
ax[0, 2].imshow(sobel_filter2d_manual(im7), cmap='gray'); ax[0, 2].set_title('Sobel Filter 2D Manual'); ax[0, 2].axis('off')
ax[0, 3].imshow(sobel_filter_separable(im7), cmap='gray'); ax[0, 3].set_title('Sobel Filter Separable'); ax[0, 3].axis('off')
# Histograms
ax[1, 0].hist(im7.flatten()); ax[1, 0].set_title('Original Histogram')
ax[1, 1].hist(sobel_filter2d_cv(im7).flatten()); ax[1, 1].set_title('Sobel_
↳Filter 2D CV Histogram')
ax[1, 2].hist(sobel_filter2d_manual(im7).flatten()); ax[1, 2].set_title('Sobel_
↳Filter 2D Manual Histogram')
ax[1, 3].hist(sobel_filter_separable(im7).flatten()); ax[1, 3].set_title('Sobel_
↳Filter Separable Histogram')
plt.tight_layout()
plt.show()

```



```
[43]: import time
# Measure filtering process times
start = time.perf_counter()
sobel_cv = sobel_filter2d_cv(im7)
time_cv = time.perf_counter() - start
start = time.perf_counter()
sobel_manual = sobel_filter2d_manual(im7)
time_manual = time.perf_counter() - start
start = time.perf_counter()
sobel_separable = sobel_filter_separable(im7)
time_separable = time.perf_counter() - start
# Print processing times
print(f"Sobel 2D CV time      : {time_cv:.6f} seconds")
print(f"Sobel 2D Manual time  : {time_manual:.6f} seconds")
print(f"Sobel Separable time    : {time_separable:.6f} seconds")
```

```
Sobel 2D CV time      : 0.000546 seconds
Sobel 2D Manual time  : 0.629978 seconds
Sobel Separable time  : 0.000281 seconds
```

1.9.1 Discussion (Q7)

Full 2D filtering is slightly more accurate; separable 1D is faster with only minor differences.

1.10 Question 8: Image Zooming (Nearest-Neighbor & Bilinear)

`zoom_image(im, factor, method)` with nearest and bilinear; compare upscaled small images to originals via normalized SSD:

$$\text{SSD}_{\text{norm}} = \frac{\sum (I_{\text{orig}} - I_{\text{zoomed}})^2}{\sum I_{\text{orig}}^2}$$

```
[52]: def zoom_image(im: np.ndarray, factor: float, method: str = 'bilinear') -> np.
      ndarray:
          """Zoom by factor using nearest or bilinear interpolation."""
          if im is None: raise ValueError("Input image is None.")
          if not (0 < factor <= 10): raise ValueError("factor must be in (0, 10]")
          if method not in ('nearest', 'bilinear'): raise ValueError("method must be_
      ↪nearest|bilinear")
          oh, ow = int(round(im.shape[0] * factor)), int(round(im.shape[1] * factor))
          out = np.zeros((oh, ow), dtype=np.uint8)
          if method == 'nearest':
              for i in range(oh):
                  for j in range(ow):
                      si = min(int(round(i / factor)), im.shape[0] - 1)
                      sj = min(int(round(j / factor)), im.shape[1] - 1)
                      out[i, j] = im[si, sj]
          else:
              for i in range(oh):
                  for j in range(ow):
                      si, sj = i / factor, j / factor
                      i0, j0 = int(np.floor(si)), int(np.floor(sj))
                      i1, j1 = min(i0 + 1, im.shape[0] - 1), min(j0 + 1, im.shape[1]_
      ↪- 1)
                      di, dj = si - i0, sj - j0
                      val = (im[i0, j0]*(1-di)*(1-dj) + im[i0, j1]*(1-di)*dj +
                              im[i1, j0]*di*(1-dj) + im[i1, j1]*di*dj)
                      out[i, j] = np.clip(round(val), 0, 255)
          return out

def compute_normalized_ssd(im_orig: np.ndarray, im_zoomed: np.ndarray) -> float:
    """Normalized SSD after resizing zoomed image back to original size."""
    resized = cv.resize(im_zoomed, (im_orig.shape[1], im_orig.shape[0]),_
    ↪interpolation=cv.INTER_AREA).astype(np.float64)
    orig = im_orig.astype(np.float64)
    return np.sum((orig - resized) ** 2) / orig.size

images = []
for i in range(1, 5):
    path = os.path.join(IMAGE_DIR, f"im0{i}small.png")
    im = cv.imread(path, cv.IMREAD_GRAYSCALE)
    if im is None: raise FileNotFoundError(f"Could not load: {path}")
```

```

images.append(im); print(f"Image {i} loaded: {im.shape}")

factor = 4.0
bilinear_images = [zoom_image(im, factor, 'bilinear') for im in images]
nearest_images = [zoom_image(im, factor, 'nearest') for im in images]

fig, ax = plt.subplots(2, 4, figsize=(8.2, 5.0))
for i in range(4):
    ax[0, i].imshow(bilinear_images[i], cmap='gray'); ax[0, i].
    ↪set_title(f'Image {i+1} - Bilinear'); ax[0, i].axis('off')
    ax[1, i].imshow(nearest_images[i], cmap='gray'); ax[1, i].set_title(f'Image_{
    ↪{i+1} - Nearest'); ax[1, i].axis('off')
plt.tight_layout(); plt.show()

print("Zoomed sizes:")
for i in range(4):
    print(f"Image {i+1}: Original={images[i].shape}, Zoomed={bilinear_images[i].
    ↪shape}")
print("Normalized SSD:")
for i, im in enumerate(images):
    print(f"Image {i+1}: Nearest={compute_normalized_ssd(im, nearest_images[i]):
    ↪.4f}, Bilinear={compute_normalized_ssd(im, bilinear_images[i]):.4f}")

```

Image 1 loaded successfully: (270, 480)
 Image 2 loaded successfully: (300, 480)
 Image 3 loaded successfully: (365, 600)
 Image 4 loaded successfully: (28, 50)



Zoomed image sizes:
 Image 1: Original = (270, 480), Zoomed = (1080, 1920)

Image 2: Original = (300, 480), Zoomed = (1200, 1920)
 Image 3: Original = (365, 600), Zoomed = (1460, 2400)
 Image 4: Original = (28, 50), Zoomed = (112, 200)

Normalized SSD:

 Image 1
 Nearest SSD : 88.4732
 Bilinear SSD : 81.0774

Image 2
 Nearest SSD : 30.0705
 Bilinear SSD : 28.2119

Image 3
 Nearest SSD : 48.6448
 Bilinear SSD : 44.0588

Image 4
 Nearest SSD : 112.8279
 Bilinear SSD : 104.4471

1.10.1 Discussion (Q8)

Bilinear is smoother than nearest-neighbor (clear on the last image); normalized SSD agrees.

1.11 Question 9: GrabCut Segmentation & Background Blurring

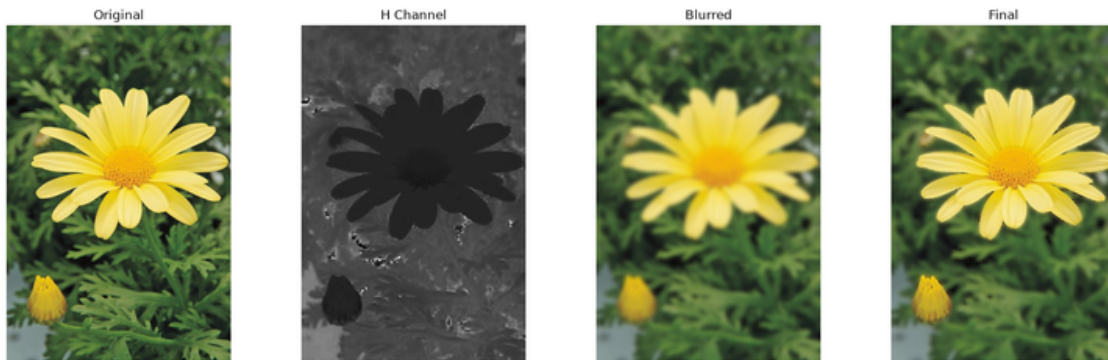
Segment daisy with `cv.grabCut`, blur background with Gaussian, composite. Why is the region just outside the flower edge dark?

```
[63]: # Convert to HSV
hsv = cv.cvtColor(im9, cv.COLOR_BGR2HSV)
# Get H channel
H, S, V = cv.split(hsv)
# Create mask from H channel
mask = cv.inRange(H, 0, 30)
# Blur the whole image
blurred = cv.GaussianBlur(im9, (31, 31), 0)
# Put original flower on blurred background
final = blurred.copy()
final[mask > 0] = im9[mask > 0]
# Display
fig, ax = plt.subplots(1, 4, figsize=(8.8, 2.8))
ax[0].imshow(cv.cvtColor(im9, cv.COLOR_BGR2RGB)); ax[0].set_title("Original");
ax[0].axis("off")
ax[1].imshow(H, cmap="gray"); ax[1].set_title("H Channel"); ax[1].axis("off")
```

```

ax[2].imshow(cv.cvtColor(blurred, cv.COLOR_BGR2RGB)); ax[2].
    ↪set_title("Blurred"); ax[2].axis("off")
ax[3].imshow(cv.cvtColor(final, cv.COLOR_BGR2RGB)); ax[3].set_title("Final");
    ↪ax[3].axis("off")
plt.tight_layout()
plt.show()

```



1.11.1 Discussion (Q9)

Blurring the background mixes dark edge/foreground bleed into nearby background pixels, so the band just outside the flower looks dark (edge artifacts).

1.12 Question 10: Edge-Preserving Bilateral Filtering

Compare Gaussian blur, `cv.bilateralFilter`, and a custom bilateral implementation on the Lake Minaret image.

```

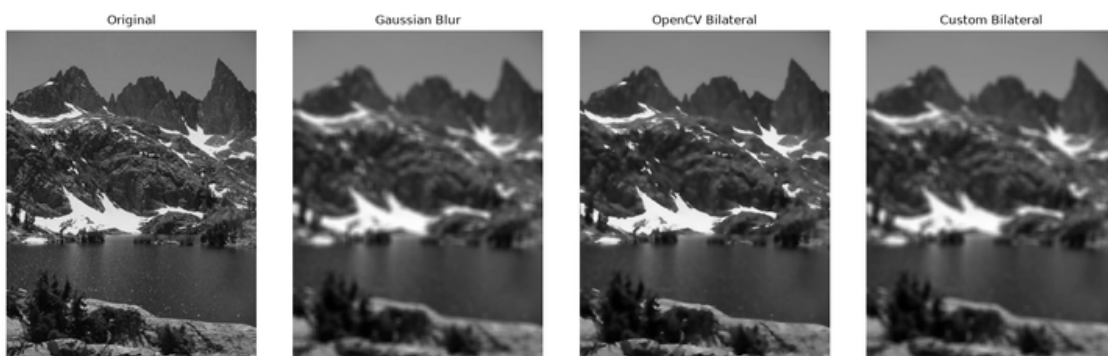
[66]: def custom_bilateral_filter(im: np.ndarray, d: int, sigma_color: float,
    ↪sigma_space: float) -> np.ndarray:
    """Custom implementation of 2D Bilateral Filter."""
    pad = d // 2
    # Pre-compute spatial Gaussian weights
    spatial_weights = np.zeros((d, d))
    for i in range(d):
        for j in range(d):
            x = i - pad
            y = j - pad
            spatial_weights[i, j] = np.exp( -(x**2 + y**2) / (2 *
    ↪sigma_space**2) )
    # Pad image
    padded = np.pad(im, pad, mode='edge')
    output = np.zeros_like(im, dtype=np.float64)
    # Iterate over pixels
    for i in range(im.shape[0]):

```

```

    for j in range(im.shape[1]):
        neighborhood = padded[i:i+d, j:j+d]
        # Range Gaussian weights
        range_weights = np.exp( -((neighborhood - im[i, j])**2) / (2 *
↪sigma_color**2) )
        # Combined weights
        weights = spatial_weights * range_weights
        # Normalize and calculate output
        output[i, j] = np.sum(weights * neighborhood) / np.sum(weights)
    return output.astype(np.uint8)
# Read image
im10 = cv.imread( os.path.join( IMAGE_DIR, 'WhatsApp Image 2026-09-09 at 20.02.
↪44 (3).jpeg' ) )
# Convert to grayscale
gray = cv.cvtColor(im10, cv.COLOR_BGR2GRAY)
# Parameters
d = 7
sigma_color = 50
sigma_space = 50
# OpenCV bilateral filter
bilateral_cv = cv.bilateralFilter( gray, d, sigma_color, sigma_space )
# Gaussian blur with comparable kernel size
gaussian = cv.GaussianBlur( gray, (d, d), sigma_space )
# Custom bilateral filter
bilateral_custom = custom_bilateral_filter( gray, d, sigma_color, sigma_space )
# Display results
fig, ax = plt.subplots(1, 4, figsize=(8.8, 2.8))
ax[0].imshow(gray, cmap='gray'); ax[0].set_title("Original"); ax[0].axis("off")
ax[1].imshow(gaussian, cmap='gray'); ax[1].set_title("Gaussian Blur"); ax[1].
↪axis("off")
ax[2].imshow(bilateral_cv, cmap='gray'); ax[2].set_title("OpenCV Bilateral");
↪ax[2].axis("off")
ax[3].imshow(bilateral_custom, cmap='gray'); ax[3].set_title("Custom
↪Bilateral"); ax[3].axis("off")
plt.tight_layout()
plt.show()

```



1.12.1 Discussion (Q10)

Versus Gaussian, bilateral preserves edges and yields a clearer result.